LAPORAN PRAKTIKUM DATABASE SISTEM PENJUALAN MENGGUNAKAN QUERY KOMPLEKS

Disusun Oleh :

Nama : Tsabita Amalia Amzani

NIM : 2502030

Kelas : TI 2B

JURUSAN TEKNIK INFORMATIKA

POLITEKNIK PURBAYA

2026

KATA PENGANTAR

Puji syukur penulis panjatkan ke hadirat Tuhan Yang Maha Esa karena atas rahmat dan karunia-Nya laporan praktikum ini dapat diselesaikan dengan baik. Laporan ini disusun untuk memenuhi tugas praktikum database mengenai pembuatan sistem penjualan sederhana menggunakan query kompleks.

Dalam laporan ini dibahas proses pembuatan database yang terdiri dari tabel pelanggan dan transaksi, serta penerapan berbagai query seperti JOIN, GROUP BY, COUNT, SUM, AVG, dan HAVING. Dengan adanya praktikum ini, diharapkan penulis dapat memahami pengolahan data menggunakan SQL dengan lebih baik.

Penulis menyadari bahwa laporan ini masih memiliki kekurangan, sehingga kritik dan saran sangat diharapkan demi perbaikan ke depannya.

Margasari, 30 Juni 2026

Penulis

DAFTAR ISI

Contents

KATA PENGANTAR ii
DAFTAR ISI iii
BAB I PENDAHULUAN 1
1.1 Latar Belakang 1
1.2 Tujuan 1
BAB II LANGKAH-LANGKAH DAN PEMBAHASAN 2
2.1 Mengakses MySQL melalui Command Prompt 2
2.2 Menampilkan Database 3
2.3 Membuat Database 3
2.4 Menggunakan Database 3
2.5 Membuat dan Menampilkan Tabel 3
2.6 Menambahkan dan Menampilkan Data 4
2.7 Query Dan Pembahasan 5
BAB III PENUTUP 9
3.1 Kesimpulan 9
3.2 Saran 9

BAB I
PENDAHULUAN

1.1 Latar Belakang

Database merupakan salah satu komponen penting dalam sistem informasi yang digunakan untuk menyimpan dan mengelola data secara terstruktur. Dalam sistem penjualan, database digunakan untuk mencatat data pelanggan serta transaksi yang dilakukan.

Penggunaan database memungkinkan data tersimpan dengan rapi, mudah diakses, dan dapat diolah menjadi informasi yang bermanfaat. Dengan menggunakan query SQL, data dapat dianalisis untuk mengetahui jumlah transaksi, total belanja, serta rata-rata pembelian pelanggan.

Oleh karena itu, dalam praktikum ini dibuat sistem database sederhana dengan dua tabel yang saling berhubungan, yaitu pelanggan dan transaksi, serta dilakukan berbagai query kompleks untuk mengolah data tersebut.

1.2 Tujuan

1. Membuat database sistem penjualan.
2. Membuat tabel pelanggan dan transaksi.
3. Menghubungkan kedua tabel dengan relasi.
4. Menginput data pelanggan dan transaksi.
5. Memahami penggunaan JOIN.
6. Memahami GROUP BY.
7. Menggunakan COUNT, SUM, AVG.
8. Menggunakan HAVING.

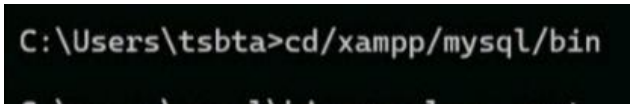
BAB II
LANGKAH-LANGKAH DAN PEMBAHASAN

2.1 Mengakses MySQL melalui Command Prompt

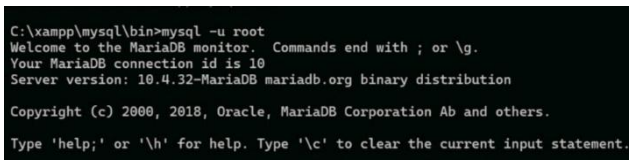
Langkah awal yang dilakukan dalam praktikum ini adalah menjalankan XAMPP Control Panel. Setelah itu, modul Apache dan MySQL diaktifkan hingga menunjukkan status Running. Proses ini bertujuan untuk memastikan bahwa server lokal telah aktif sehingga MySQL dapat digunakan tanpa kendala.



Setelah XAMPP aktif, langkah selanjutnya adalah membuka aplikasi Command Prompt. Kemudian pengguna mengetikkan perintah untuk masuk ke direktori MySQL dengan perintah cd/xampp/mysql/bin. Perintah ini digunakan agar sistem dapat mengenali file eksekusi MySQL.



Selanjutnya, dilakukan proses login ke MySQL dengan mengetikkan perintah mysql -u root. Perintah ini digunakan untuk masuk ke dalam sistem MySQL sebagai pengguna root yang memiliki hak akses penuh terhadap database.



2.2 Menampilkan Database

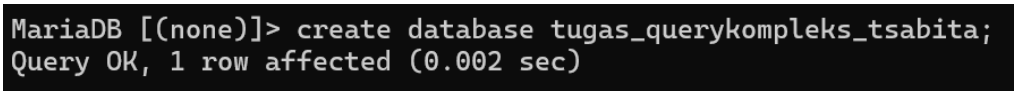
Setelah berhasil masuk ke MySQL, pengguna dapat melihat daftar database yang tersedia dengan menggunakan perintah show databases;. Perintah ini menampilkan semua database yang ada pada sistem, baik database bawaan maupun database yang telah dibuat sebelumnya.



```
MariaDB [(none)]> show databases;
+----------------------------+
| Database                   |
+----------------------------+
| information_schema         |
| mysql                      |
| performance_schema         |
| pertemuan11_aggregate      |
| pertemuan12_grouphy        |
| pertemuan13_having         |
| pertemuan14_querykompleks  |
| phpmyadmin                 |
| test                       |
| toko_buku                  |
| tokobuku_diera             |
| tugas11_aggregate_tsabita  |
| tugas_groupby_tsabita      |
| tugas_having_tsabita       |
+----------------------------+
14 rows in set (0.072 sec)
```

2.3 Membuat Database

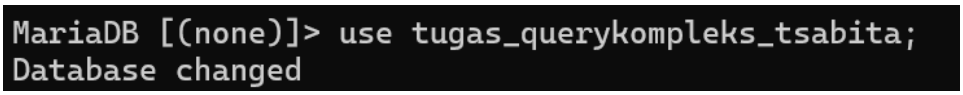
Langkah berikutnya adalah membuat database baru dengan perintah create database nama_database;. Database ini akan digunakan sebagai tempat penyimpanan data yang berkaitan dengan database yang dibuat.



```
MariaDB [(none)]> create database tugas_querykompleks_tsabita;
Query OK, 1 row affected (0.002 sec)
```

2.4 Menggunakan Database

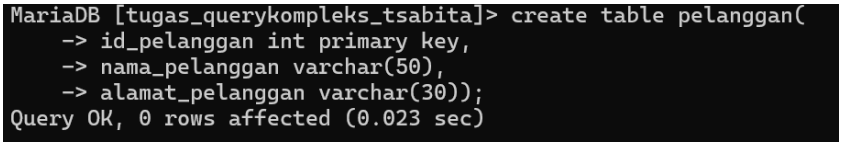
Untuk mulai bekerja dengan database yang telah dibuat, digunakan perintah use nama_database;. Perintah ini berfungsi untuk mengaktifkan database sehingga pengguna dapat membuat tabel dan mengelola data di dalamnya.



```
MariaDB [(none)]> use tugas_querykompleks_tsabita;
Database changed
```

2.5 Membuat dan Menampilkan Tabel

Setelah database aktif, langkah selanjutnya adalah membuat tabel dengan perintah create table nama_tabel(dan diakhiri dengan);. Tabel ini dibuat dengan beberapa kolom seperti nomor id, nama, alamat, dan laim-lain. Setiap kolom memiliki tipe data tertentu sesuai kebutuhan. Pembuatan tabel ini bertujuan untuk menyimpan data secara terstruktur sehingga mudah untuk diakses dan dikelola.



```
MariaDB [tugas_querykompleks_tsabita]> create table pelanggan(
    -> id_pelanggan int primary key,
    -> nama_pelanggan varchar(50),
    -> alamat_pelanggan varchar(30));
Query OK, 0 rows affected (0.023 sec)
```

```
MariaDB [tugas_querykompleks_tsabita]> create table transaksi(
    -> id_transaksi int primary key,
    -> id_pelanggan int,
    -> total_belanja int);
Query OK, 0 rows affected (0.022 sec)
```

Untuk melihat struktur tabel yang telah dibuat, digunakan perintah desc nama_tabel;. Perintah ini menampilkan informasi mengenai kolom, tipe data, serta keterangan lainnya.

```
MariaDB [tugas_querykompleks_tsabita]> desc pelanggan;
+------------------+-------------+------+-----+---------+-------+
| Field            | Type        | Null | Key | Default | Extra |
+------------------+-------------+------+-----+---------+-------+
| id_pelanggan     | int(11)     | NO   | PRI | NULL    |       |
| nama_pelanggan   | varchar(50) | YES  |     | NULL    |       |
| alamat_pelanggan | varchar(30) | YES  |     | NULL    |       |
+------------------+-------------+------+-----+---------+-------+
3 rows in set (0.033 sec)
```

```
MariaDB [tugas_querykompleks_tsabita]> desc transaksi;
+---------------+---------+------+-----+---------+-------+
| Field         | Type    | Null | Key | Default | Extra |
+---------------+---------+------+-----+---------+-------+
| id_transaksi  | int(11) | NO   | PRI | NULL    |       |
| id_pelanggan  | int(11) | YES  |     | NULL    |       |
| total_belanja | int(11) | YES  |     | NULL    |       |
+---------------+---------+------+-----+---------+-------+
3 rows in set (0.026 sec)
```

2.6 Menambahkan dan Menampilkan Data

Data dimasukkan ke dalam tabel menggunakan perintah insert into nama_tabel values (dan diakhiri dengan);. Proses ini bertujuan untuk mengisi tabel dengan data yang diperlukan.

Setelah proses pembuatan tabel dan penginputan data selesai dilakukan, langkah selanjutnya adalah menampilkan data yang telah tersimpan di dalam tabel. Proses ini bertujuan untuk memastikan bahwa data yang dimasukkan telah berhasil tersimpan dengan benar di dalam database. Untuk menampilkan seluruh data ketik perintah select * from nama_tabel;.

```
MariaDB [tugas_querykompleks_tsabita]> insert into pelanggan values
    -> (1,'Tsabita','Jakarta'),
    -> (2,'Dinda','Tegal'),
    -> (3,'Serly','Cirebon'),
    -> (4,'Rekhan','Surabaya'),
    -> (5,'Dimas','Bandung'),
    -> (6,'Dewi','Brebes'),
    -> (7,'Lia','Purwokerto'),
    -> (8,'Zilda','Bekasi'),
    -> (9,'Lana','Solo'),
    -> (10,'Rizka','Yogyakarta');
Query OK, 10 rows affected (0.062 sec)
Records: 10  Duplicates: 0  Warnings: 0

MariaDB [tugas_querykompleks_tsabita]> select * from pelanggan;
+--------------+----------------+------------------+
| id_pelanggan | nama_pelanggan | alamat_pelanggan |
+--------------+----------------+------------------+
|            1 | Tsabita        | Jakarta          |
|            2 | Dinda          | Tegal            |
|            3 | Serly          | Cirebon          |
|            4 | Rekhan         | Surabaya         |
|            5 | Dimas          | Bandung          |
|            6 | Dewi           | Brebes           |
|            7 | Lia            | Purwokerto       |
|            8 | Zilda          | Bekasi           |
|            9 | Lana           | Solo             |
|           10 | Rizka          | Yogyakarta       |
+--------------+----------------+------------------+
10 rows in set (0.010 sec)
```

```
MariaDB [tugas_querykompleks_tsabita]> insert into transaksi values
    -> (1,1,300000),
    -> (2,1,350000),
    -> (3,2,200000),
    -> (4,2,500000),
    -> (5,2,150000),
    -> (6,3,100000),
    -> (7,4,600000),
    -> (8,5,250000),
    -> (9,5,400000),
    -> (10,5,100000),
    -> (11,6,500000),
    -> (12,7,550000),
    -> (13,8,250000),
    -> (14,9,150000),
    -> (15,10,300000);
Query OK, 15 rows affected (0.009 sec)
Records: 15  Duplicates: 0  Warnings: 0

MariaDB [tugas_querykompleks_tsabita]> select * from transaksi;
+--------------+--------------+---------------+
| id_transaksi | id_pelanggan | total_belanja |
+--------------+--------------+---------------+
|            1 |            1 |        300000 |
|            2 |            1 |        350000 |
|            3 |            2 |        200000 |
|            4 |            2 |        500000 |
|            5 |            2 |        150000 |
|            6 |            3 |        100000 |
|            7 |            4 |        600000 |
|            8 |            5 |        250000 |
|            9 |            5 |        400000 |
|           10 |            5 |        100000 |
|           11 |            6 |        500000 |
|           12 |            7 |        550000 |
|           13 |            8 |        250000 |
|           14 |            9 |        150000 |
|           15 |           10 |        300000 |
+--------------+--------------+---------------+
15 rows in set (0.001 sec)
```

2.7 Query Dan Pembahasan

1. JOIN

Pada query pertama digunakan perintah JOIN untuk menggabungkan data dari dua tabel yaitu tabel pelanggan dan tabel transaksi. JOIN dilakukan berdasarkan relasi antara kolom id pada tabel pelanggan dengan id_pelanggan pada tabel transaksi.

```
MariaDB [tugas_querykompleks_tsabita]> select nama_pelanggan, total_belanja
    -> from pelanggan
    -> join transaksi
    -> on pelanggan.id_pelanggan=transaksi.id_pelanggan;
+----------------+--------------+
| nama_pelanggan | total_belanja |
+----------------+--------------+
| Tsabita        |       300000 |
| Tsabita        |       350000 |
| Dinda          |       200000 |
| Dinda          |       500000 |
| Dinda          |       150000 |
| Serly          |       100000 |
| Rekhan         |       600000 |
| Dimas          |       250000 |
| Dimas          |       400000 |
| Dimas          |       100000 |
| Dewi           |       500000 |
| Lia            |       550000 |
| Zilda          |       250000 |
| Lana           |       150000 |
| Rizka          |       300000 |
+----------------+--------------+
15 rows in set (0.013 sec)
```

Dengan menggunakan JOIN, data yang sebelumnya terpisah dapat ditampilkan menjadi satu kesatuan. Hasil dari query ini menampilkan nama pelanggan beserta total belanja yang dilakukan pada setiap transaksi. Query ini sangat berguna untuk melihat data transaksi secara detail berdasarkan pelanggan yang melakukan pembelian.

2. JOIN + GROUP BY

Pada query ini digunakan JOIN untuk menggabungkan tabel pelanggan dan transaksi berdasarkan kolom id dan id_pelanggan. Setelah data digabungkan, dilakukan pengelompokan menggunakan GROUP BY berdasarkan nama pelanggan.

```
MariaDB [tugas_querykompleks_tsabita]> select nama_pelanggan,
    -> count(*) as jumlah_transaksi
    -> from pelanggan
    -> join transaksi
    -> on pelanggan.id_pelanggan=transaksi.id_pelanggan
    -> group by nama_pelanggan;
+----------------+-----------------+
| nama_pelanggan | jumlah_transaksi |
+----------------+-----------------+
| Dewi           |               1 |
| Dimas          |               3 |
| Dinda          |               3 |
| Lana           |               1 |
| Lia            |               1 |
| Rekhan         |               1 |
| Rizka          |               1 |
| Serly          |               1 |
| Tsabita        |               2 |
| Zilda          |               1 |
+----------------+-----------------+
10 rows in set (0.009 sec)
```

Selanjutnya fungsi COUNT() digunakan untuk menghitung jumlah transaksi pada setiap pelanggan. Hasil dari query ini menampilkan berapa kali masing-masing pelanggan melakukan transaksi, sehingga dapat diketahui pelanggan yang paling aktif.

3. COUNT + JOIN + GROUP BY

Pada query ini digunakan fungsi COUNT() untuk menghitung jumlah transaksi berdasarkan alamat pelanggan. Data dari tabel pelanggan dan transaksi terlebih dahulu

digabungkan menggunakan JOIN agar informasi alamat dan transaksi dapat ditampilkan secara bersamaan.

```
MariaDB [tugas_querykompleks_tsabita]> select alamat_pelanggan,
    -> count(*) as jumlah_transaksi
    -> from pelanggan
    -> join transaksi
    -> on pelanggan.id_pelanggan=transaksi.id_pelanggan
    -> group by alamat_pelanggan;
+------------------+------------------+
| alamat_pelanggan | jumlah_transaksi |
+------------------+------------------+
| Bandung          |                3 |
| Bekasi           |                1 |
| Brebes           |                1 |
| Cirebon          |                1 |
| Jakarta          |                2 |
| Purwokerto       |                1 |
| Solo             |                1 |
| Surabaya         |                1 |
| Tegal            |                3 |
| Yogyakarta       |                1 |
+------------------+------------------+
10 rows in set (0.001 sec)
```

Setelah data digabungkan, dilakukan pengelompokan menggunakan GROUP BY berdasarkan kolom alamat. Dengan demikian, semua pelanggan yang memiliki alamat yang sama akan dikelompokkan dalam satu kategori.

4. SUM + JOIN + GROUP BY

Pada query ini digunakan fungsi SUM() untuk menghitung total keseluruhan belanja dari setiap pelanggan. Data transaksi yang telah digabungkan dengan data pelanggan kemudian dikelompokkan berdasarkan nama pelanggan.

```
MariaDB [tugas_querykompleks_tsabita]> select nama_pelanggan,
    -> sum(total_belanja) as jumlah_pembelian
    -> from pelanggan
    -> join transaksi
    -> on pelanggan.id_pelanggan=transaksi.id_pelanggan
    -> group by nama_pelanggan;
+---------------+-----------------+
| nama_pelanggan | jumlah_pembelian |
+---------------+-----------------+
| Dewi          |          500000 |
| Dimas         |          750000 |
| Dinda         |          850000 |
| Lana          |          150000 |
| Lia           |          550000 |
| Rekhan        |          600000 |
| Rizka         |          300000 |
| Serly         |          100000 |
| Tsabita       |          650000 |
| Zilda         |          250000 |
+---------------+-----------------+
10 rows in set (0.001 sec)
```

Setelah proses pengelompokan, fungsi SUM akan menjumlahkan seluruh nilai total belanja pada setiap kelompok. Hasil dari query ini menunjukkan total uang yang telah dibelanjakan oleh masing-masing pelanggan. Query ini sangat membantu dalam mengetahui pelanggan dengan nilai pembelian terbesar.

5. AVG + JOIN + GROUP BY

Pada query ini digunakan fungsi AVG() untuk menghitung rata-rata nilai transaksi dari setiap pelanggan. Data yang telah digabungkan kemudian dikelompokkan berdasarkan nama pelanggan.

```
MariaDB [tugas_querykompleks_tsabita]> select nama_pelanggan,
    -> avg(total_belanja) as ratarata_belanja
    -> from pelanggan
    -> join transaksi
    -> on pelanggan.id_pelanggan=transaksi.id_pelanggan
    -> group by nama_pelanggan;
+----------------+------------------+
| nama_pelanggan | ratarata_belanja |
+----------------+------------------+
| Dewi           |      500000.0000 |
| Dimas          |      250000.0000 |
| Dinda          |      283333.3333 |
| Lana           |      150000.0000 |
| Lia            |      550000.0000 |
| Rekhan         |      600000.0000 |
| Rizka          |      300000.0000 |
| Serly          |      100000.0000 |
| Tsabita        |      325000.0000 |
| Zilda          |      250000.0000 |
+----------------+------------------+
10 rows in set (0.007 sec)
```

Fungsi AVG akan menghitung rata-rata dari total belanja dalam setiap kelompok. Hasil dari query ini dapat digunakan untuk mengetahui kebiasaan belanja pelanggan, apakah cenderung melakukan pembelian dalam jumlah kecil atau besar secara rata-rata.

6. JOIN + GROUP BY + HAVING

A. Berdasarkan Total Pembelian

```
MariaDB [tugas_querykompleks_tsabita]> select nama_pelanggan,
    -> sum(total_belanja) as total_pembelian
    -> from pelanggan
    -> join transaksi
    -> on pelanggan.id_pelanggan=transaksi.id_pelanggan
    -> group by nama_pelanggan
    -> having sum(total_belanja)>500000;
+----------------+-----------------+
| nama_pelanggan | total_pembelian |
+----------------+-----------------+
| Dimas          |          750000 |
| Dinda          |          850000 |
| Lia            |          550000 |
| Rekhan         |          600000 |
| Tsabita        |          650000 |
+----------------+-----------------+
5 rows in set (0.008 sec)
```

Pada query ini digunakan klausa HAVING untuk menyaring hasil dari GROUP BY. Setelah data dikelompokkan dan dihitung total belanjanya menggunakan SUM, kemudian dilakukan penyaringan hanya pada pelanggan yang memiliki total belanja lebih dari 500.000.

B. Berdasarkan Total Transaksi

```
MariaDB [tugas_querykompleks_tsabita]> select nama_pelanggan,
    -> count(*) as total_transaksi
    -> from pelanggan
    -> join transaksi
    -> on pelanggan.id_pelanggan=transaksi.id_pelanggan
    -> group by nama_pelanggan
    -> having count(*)>=2;
+----------------+-----------------+
| nama_pelanggan | total_transaksi |
+----------------+-----------------+
| Dimas          |               3 |
| Dinda          |               3 |
| Tsabita        |               2 |
+----------------+-----------------+
3 rows in set (0.001 sec)
```

Pada query ini digunakan kombinasi COUNT dan HAVING untuk menampilkan pelanggan yang memiliki jumlah transaksi minimal 2 kali. Data terlebih dahulu digabungkan dan dikelompokkan berdasarkan nama pelanggan.

Setelah itu, fungsi COUNT digunakan untuk menghitung jumlah transaksi, kemudian HAVING digunakan untuk memfilter hasil agar hanya menampilkan pelanggan dengan transaksi minimal 2 kali. Query ini berguna untuk mengetahui pelanggan yang aktif dalam melakukan pembelian.

C. Berdasarkan Rata-Rata

```
MariaDB [tugas_querykompleks_tsabita]> select nama_pelanggan,
    -> avg(total_belanja) as ratarata_pembelian
    -> from pelanggan
    -> join transaksi
    -> on pelanggan.id_pelanggan=transaksi.id_pelanggan
    -> group by nama_pelanggan
    -> having avg(total_belanja)>300000;
+----------------+--------------------+
| nama_pelanggan | ratarata_pembelian |
+----------------+--------------------+
| Dewi           |        500000.0000 |
| Lia            |        550000.0000 |
| Rekhan         |        600000.0000 |
| Tsabita        |        325000.0000 |
+----------------+--------------------+
4 rows in set (0.001 sec)
```

Pada query ini digunakan fungsi AVG() untuk menghitung rata-rata nilai transaksi setiap pelanggan. Data yang telah digabungkan kemudian dikelompokkan berdasarkan nama pelanggan.

Setelah itu, fungsi AVG digunakan untuk menghitung rata-rata belanja. Dengan menggunakan HAVING, hanya pelanggan yang memiliki rata-rata belanja lebih dari 300.000 yang ditampilkan. Query ini membantu mengetahui pelanggan yang cenderung melakukan pembelian dalam jumlah besar secara konsisten.

BAB III
PENUTUP

3.1 Kesimpulan

Dari praktikum ini dapat disimpulkan bahwa database sangat membantu dalam pengelolaan data sistem penjualan. Dengan dua tabel yang saling berhubungan, data pelanggan dan transaksi dapat diorganisir dengan baik.

Penggunaan JOIN memungkinkan penggabungan data, sedangkan GROUP BY dan fungsi agregat membantu dalam analisis data seperti jumlah transaksi, total belanja, dan rata-rata pembelian. HAVING digunakan untuk menyaring hasil analisis tersebut.

Dengan memahami query kompleks, pengolahan data menjadi lebih efektif dan informatif.

3.2 Saran

Dalam pembuatan database, struktur tabel harus dirancang dengan baik agar relasi antar data jelas dan efisien. Selain itu, penting untuk memahami penggunaan query SQL secara bertahap mulai dari yang sederhana hingga kompleks.

Disarankan untuk sering berlatih menggunakan berbagai kombinasi query agar kemampuan dalam pengolahan data semakin meningkat. Pengembangan sistem yang lebih kompleks juga dapat dilakukan untuk menambah wawasan.